ARQUITECTURA DE SOFTWARE

1 POR CAPAS

Forma estructurada de organizar un programa colocandolas en niveles jerárquicos superpuestos, donde cada capa cumple una tarea específica y se comunica de manera unidireccional

CARACTERISTICAS CLAVE

1. Independencia jerárquica y modularidad estricta.
2. Abstracción de los detalles de bajo nivel respecto a la lógica de negocio superior

VENTAJAS

Mantenimiento simplificado, excelente para trabajo en equipo modular, alta reutilización de código y escalabilidad independiente de las capas

DESVENTAJAS

Posible sobrecarga de rendimiento por el paso de llamadas a través de múltiples capas y exceso de complejidad para aplicaciones demasiado pequeñas

PATRONES DE DISEÑO

SINGLETON
Garantiza que una clase tenga una única instancia en todo el ciclo de vida de la aplicación y proporciona un punto de acceso global a ella; ideal para compartir recursos pesados en proyectos grandes

FACTORY METHOD
Define una interfaz para crear un objeto, pero deja que las subclases decidan qué clase concreta instanciar; útil cuando un programa necesita gestionar múltiples creaciones de objetos dinámicamente

2 MONOLITICA

Modelo de diseño donde toda la aplicacion se construye como una unidad de codigo unico

ORGANIZACION

1. Usuario
2. Presentacion
3. Logica de negocio
4. Acceso a datos
5. Base de datos

VENTAJAS

Simplicidad inicial, despliegue unificado y pruebas directas l tener todo en un solo bloque

DESVENTAJAS

Escalabilidad limitada, alto riego ante fallos y dificultad de mantenimineto al crecer

PATRONES DE DISEÑO

ABSTRACT FACTORY
Crea familias de objetos relacionados o dependientes sin especificar sus clases concretas; garantiza compatibilidad en proyectos grandes

BUILDER
Permite la creación de objetos complejos paso a paso, separando el algoritmo de construcción de su representación final

PROTOTYPE
Crea nuevos objetos clonando instancias existentes (prototipos) para evitar costos altos de inicialización directa

3 CLIENTE SERVIDOR

En este modelo los roles están claramente separados: el Cliente solicita recursos, servicios o procesamiento, y el Servidor procesa la petición, interactúa con la base de datos y devuelve una respuesta estructurada

ORGANIZACION

- 2 Capas: Interfaz de usuario conectada directamente a la base de datos en la nube.
- 3 Capas: Separación física/lógica de Cliente, Servidor de Aplicaciones y Base de Datos.
- N Capas: Servicios independientes distribuidos para cada nivel de lógica o negocio específica.

VENTAJAS

Centralización de datos y seguridad, administración simplificada en el servidor, fácil mantenimiento y soporte de múltiples plataformas

DESVENJATAS

Alta dependencia del servidor (un fallo de infraestructura afecta a toda la red) y costos elevados de configuración técnica

PATRONES DE DISEÑO

ADAPTER
Funciona como un traductor o puente en el backend para permitir que clases o sistemas con interfaces incompatibles trabajen juntos sin alterar su código fuente.

FACADE
Proporciona una interfaz unificada y simplificada para un conjunto de subsistemas complejos, reduciendo el acoplamiento con el cliente

4 ARQUITECTURA MVC

Patrón arquitectónico clásico que desacopla la interfaz de usuario de la lógica interna en tres componentes independientes con responsabilidades únicas

ORGANIZACION

- Modelo: Gestiona los datos, el estado de la aplicación y la lógica de negocio esencial.
- Vista: Representa la interfaz gráfica y los elementos visuales con los que interactúa el usuario (Frontend).
- Controlador: Actúa como intermediario gestionando las entradas del usuario, actualizando el modelo y respondiendo con la vista adecuada

VENTAJAS

Excelente organización del código, permite que desarrolladores trabajen en paralelo (modelo, vista o controlador) sin interferir y garantiza una alta escalabilidad en aplicaciones medianas y grandes

DESVENTAJAS

Incremento considerable en la cantidad de archivos y necesidad rigurosa de estructurar las conexiones para evitar acoplamientos indeseados

PATRONES DE DISEÑO

DECORATOR
Permite añadir nuevas funcionalidades a un objeto de forma dinámica colocándolo dentro de objetos contenedores especiales, sin alterar el código original de la clase

PROXY
Proporciona un objeto sustituto o intermediario que controla el acceso al objeto real (por ejemplo, verificando permisos de seguridad del usuario antes de permitir una acción).

BRIDGE
Desacopla una abstracción de su implementación para que ambas puedan variar y evolucionar de forma totalmente independiente